Int. j. inf. tecnol. (February 2024) 16(2):731–743
https://doi.org/10.1007/s41870-023-01642-8

ORIGINAL RESEARCH

An intent recognition pipeline for conversational AI

C. B. Chandrakala[1] · Rohit Bhardwaj[1] · Chetana Pujari[1]

Received: 7 February 2023 / Accepted: 25 October 2023 / Published online: 29 December 2023

Abstract Natural Language Processing (NLP) is one of the Artificial Intelligence applications that is entitled to allow computers to process and understand human language. These models are utilized to analyze large volumes of text and also support aspects like text summarization, language translation, context modeling, and sentiment analysis. Natural language, a subset of Natural Language Understanding (NLU), turns natural language into structured data. NLU accomplishes intent classification and entity extraction. The paper focuses on a pipeline to maximize the coverage of a conversational AI (chatbot) by extracting maximum meaningful intents from a data corpus. A conversational AI can best answer queries with respect to the dataset if it is trained on the maximum number of intents that can be gathered from the dataset which is what we focus on getting in this paper. The higher the intent we gather from the dataset, the more of the dataset we cover in training the conversational AI. The pipeline is modularized into three broad categories - Gathering the intents from the corpus, finding misspellings and synonyms of the intents, and finally deciding the order of intents to be picked up for training any classifier ML model. Several heuristic and machine-learning approaches have been considered for optimum results. For finding misspellings and synonyms, they are extracted through text vector neural network-based algorithms. Then the system concludes with a suggestive priority list of intents that should be fed to a classification model. In the end, an example of three intents from the corpus is picked, and their order is suggested for the optimum functioning of the pipeline. This paper attempts to pick intents in descending order of their coverage in the corpus in the most optimal way possible.

Keywords Chatbot · Intent detection · Machine learning · Neural network · Word embeddings · Sentence embeddings

1 Introduction

The world of customer service is transforming. Earlier, we had humans (agents) behind telephones trying their best to answer diverse queries of customers. This was manual and tedious. The scope of misinformation between the agents and customers was high as well. Apart from that, companies had to place humans behind telephones 24/7 to answer queries coming from different time zones. As Artificial Intelligence has disrupted several industries, it has disrupted the customer service industry as well. AI-powered chatbots provide 24/7 customer support, answering frequently asked questions and providing support without the need for human intervention. This helps agents to handle more complex inquiries or provide the empathy and personal touch that a chatbot won't provide.

A chatbot uses the concept of NLP under the hood of Artificial Intelligence to communicate with a human. Used across almost every business domain today, for example - E-Commerce, Real Estate, Healthcare, Banking, Financial Services, and Insurance Sector, etc. by Mohit [1] and Fernandes [2] It is possible to train these bots to interact in multiple languages and around the clock.

✉ Chetana Pujari
chetana.pujari@manipal.edu

C. B. Chandrakala
chandrakala.cb@manipal.edu

Rohit Bhardwaj
rohit.bhardwaj434@gmail.com

[1] Department of Information and Communication Technology, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka 576104, India

NLU is a part of NLP that converts language into statistical data. Intent classification and entity extraction are the primary reasons why we need NLU. When a sentence is read, it understands the meaning of the sentence. What a user is trying to convey or accomplish in a sentence is called Intent. An NLU model is fed with data that provides the mapping of sentences to their respective intents. After getting trained, the model classifies a new sentence into one of the intents that it was trained on. Entity extraction is used to recognize key pieces of knowledge in the corpus of text. Things like the time, place, and name of a person all provide additional context and information related to intent as mentioned by Blei et al. [3]. Intent classification and entity extraction are the goals achieved by conversational AI.

Identifying intents and training a classifier model is challenging in a huge corpus of data [4]. Identifying intents requires annotators to tag every query of the corpus with specific intent. Annotators must devote their time and energy to this tedious task. This increases the operational overhead of training a classifier model and introduces human bias into intent tagging. Gold labels, i.e. tagged queries by humans considered to be ground truth, will never be accurate if the human is unfamiliar with the business domain.

This paper attempts to reduce the overhead associated with intent identification. As intents are hard to recognize with the naked eye and there is no set technique for identifying them in the corpus in the most efficient manner, it is difficult to quantify the overhead. This paper by Bhardwaj [5] aims to cover as many intents as possible in a short period of time using an exploration pipeline that combines machine learning with heuristics to cover as many intents as possible.

The objectives of the paper are -

1. To extract maximum meaningful intents from the corpus.
2. Prioritizing the order of intents to be picked up for coverage boost.
3. Reducing the operational overhead before training a classifier model in the most optimum way.

The rest of the paper is organized as follows. Sect. 2 discusses a related study. As an example, Sect. 3 discusses the data corpus used to build the pipeline, Sect. 4 discusses how the pipeline was constructed, Sect. 5 describes the entire pipeline, Sect. 6 describes the evaluation process, and Sect. 7 concludes the paper.

2 Related work

The research works in the past outline Conversational AI or Human Machine Dialogue Interaction Systems, but they do not focus on intent extraction in specific. We are targeting achieving intents fast and right in this paper via a pipeline that consists of both machine learning and heuristic-based techniques.

Intent Detection is demonstrated as a classification problem in the paper presented by Tran et al. [6]. In the paper, the authors have taken intent detection as a query belonging to a specific intent predefined by the user. The dataset used is refined and they have avoided feature engineering. Their whole approach to intent detection has been using bi-LSTM and CNN neural network models. Bi-LSTMs are computationally intensive to train and need lots of data. If they are trained on short customer support queries, there is a good chance they may miss-classify intents of long customer support queries. CNNs, on the other hand, are detailed oriented and to get accurate results on the dataset, need to be regularly trained. If we ensemble both bi-LSTM and CNN models together we will increase the model training time where we may need a larger corpus of filtered data and finding the hyperparameter tuning for both models can be time-consuming. However, in this paper, we have considered an unfiltered dataset, included feature engineering, and focused on creating a pipeline, involving both heuristic and ML-based techniques, that do not need much training data as well as hyperparameter tuning, for intent extraction in such unfiltered datasets at a quick pace. We have focused on covering the maximum amount of the dataset through the intents that we extract and even the precedence order of picking those intents to train a classifier model on the dataset. Our paper also focuses on identifying multiple intents present in the same query sentence. For example - I want to return my order and ask for a refund - has two intents - return and refund.

A comparison between different intent detection schemes has been researched in the work proposed by Liu et al. [7]. They have covered various deep learning techniques to emulate the heuristic-based techniques in terms of multi-intent detection. They have shown how a capsule network model, that overcomes the representational shortcomings of CNN, shows superior results for intent detection as well as multi-intent detection. But it's unclear from their description how well the model scales to different business domains or if it can handle a wide variety of intents and contexts effectively. Their method needs a large and diverse dataset for the capsule to train on. Company 'A' can have different queries from Company 'B'; hence we cannot use the same model trained on A to identify intents from the dataset of B. Therefore, in this paper, an attempt has been made to highlight the solution that lies somewhere between deep learning and traditional statistics-based approaches. A common pipeline has been proposed in this paper to identify intents in a data corpus irrespective of the business domain. The intent classification method based on word embedding has better

representational ability is domain-extensible for different classification contents and has been used in the pipeline.

In this paper by Dzikiene et al. [8], deep neural network architecture and hyperparameter values were tuned. They have gone ahead and shown that the BERT multilingual vectorization with the CNN classifier was proven to be a good choice for intent detection for their datasets. The hyperparameter tuning for their models was done by Random Search as well as the Tree-Structured Parzen Estimator (TPE). The major drawback of Random Search is it does not use previous iterations as guidance to the search due to which we may be spending time training the model based on parameters that may never yield an optimum result. On the other hand, TPE has two phases - a warm-up phase and an optimization phase. In the warmup phase, it randomly tries different hyperparameter values to build a model and evaluates the model's performance on the validation data. In the optimization phase, it divides hyperparameter combinations into "good" and "bad" groups. It then focuses on exploring combinations in the "good" group and refines the search based on Bayesian rules to find the best hyperparameters. TPE is more complex and computationally intensive than random search due to its Bayesian optimization process. This complexity can make it slower. TPE relies on the initial warm-up phase, which can consume additional computational resources and time. TPE's effectiveness can depend on the configuration of its hyperparameters. Selecting the right configuration is crucial for its success. For both these hyperparameter tuning techniques, they do not guarantee finding the global optimum (the absolute best hyperparameters). Spending much time in hyper-parameter tuning would delay us in intent detection later. To overcome these shortcomings, we can rely on statistical-based methods that do not need hyperparameter tuning or clustering methods based on word embeddings that do not need much hyperparameter tuning, which we have involved in the initial stages of our pipeline.

A clustering approach is used in the work by Kathuria et al. [9] and Jansen et al. [10]. The work has clustered several search engine queries that are categorized as informational (user asks for information regarding a particular topic), navigational (user wants to navigate web pages), or transactional (user asks queries that are noninformation) using a k-means clustering approach based on a variety of query traits. This work has not considered the fact that a single query can belong to more than one intent, as our proposed work has (multi-intent queries), and they do not know whether this approach would yield the same results in other data sets. According to Jansen and Spink (2005) as presented by Ratner et al. [11] and [12], the approach we have considered in building the pipeline would yield the same results on other datasets as well.

Amneth et al., in their study, cited as [13], delved into the realm of social media data analysis, with a particular focus on journalism, where they explored the impact of Non-Performing Loans (NPLs). They examined how NPLs influence the generation of aggregated social news. Shin et al., as referenced in [14], introduced a novel approach aimed at mitigating ambiguity in Myanmar text. Their method employed techniques such as word segmentation and POS tagging to address translation bottlenecks.

Arepalli et al., as detailed in [15], contributed to the field of sentiment analysis. They computed sentiment scores on social media data and applied opinion mining-based classification methods to gain insights into the sentiment expressed in the data. Sintayehu et al., in their work cited as [16], conducted an evaluation of named entity resources. They analyzed the performance and made determinations regarding the effectiveness of these resources. Anjali et al., as mentioned in [17], undertook the task of mapping clinical narratives to a knowledge graph. This mapping has the potential to be used in healthcare applications for informed decision-making.

An ensembling of classifiers to combine syntactic and semantic features to effectively detect the user's intention is proposed by Figueroa et al. [18]. These queries were manually annotated, first extracting a sample of 30,000 random queries. In our paper, we do not need annotators and manual tagging of data to check the accuracy, as we have not used supervised learning approaches. This reduces manual effort and saves time, and we do not need to calculate cross-annotator annotation accuracy with techniques like the Borda Count Method, Copeland's method, etc.

The article on string similarity referred from the paper cited at [1] shows the most commonly used string similarity methods, but does not consider a specific use case or metric improvement to be taken into account. In our paper, we focus on coverage as a metric. The maximum information we can extract from the dataset by recognizing the maximum intents from the dataset.

The article by Xu et al. [19] describes various topic modeling concepts in detail but does not provide an appropriate parameter tuning technique in the bid to get the improvement in coverage while maintaining a good accuracy as well.

In some cases, a query can have two or more intents, and it is important to identify such queries where research has not been much but has important business implications. As part of this paper, a level of the pipeline is considered.

3 Data corpus

Data corpus is the main requirement for intent recognition pipelines. In order to achieve the best-fit dataset, a method of data collection and text mining is adopted as shown in Fig. 1.

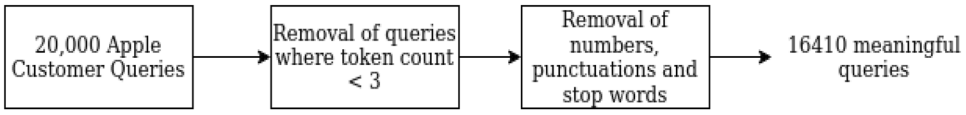

Fig. 1 Extracting meaningful queries from the dataset

1. Data collection - To get user queries, customer tweets are obtained from the Kaggle dataset of Customer Support on Twitter [20]. These tweets consist of user complaints tagging the official handles of several companies. From the repository, the proposed work extracts the tweets regarding Apple Support, the official handle for the company - Apple.
2. Text mining - The operations carried under it are -
   - The queries with less than 3 words usually contain salutations like - 'Thank You', 'Please', 'Okay', 'Welcome', 'Pleasure', 'Hi', 'Hello' etc, or other forms of noise. Hence, they are removed.
   - Numbers, Punctuations, and Stop Words (eg. I, the, yours, is, etc) are also removed from the queries as they contribute to the noise in a query.

The count of tweets decreased from 20,000 to 16,000 after the text mining techniques.

4 Building of intent recognition pipeline

The pipeline can be broken down into the following modules and the process followed is presented in the algorithm 1:

1. Token based intent extraction
2. Semantic-based intent extraction
3. Maximizing coverage
4. Priority of intent pickup

Algorithm 1 An algorithm depicting the proposed work

Input
$data_{corpus}$
Output
$Training_{set} = Max_{coverage}(data_{corpus})$
Steps
for do $word$ *in* $data_{corpus}$
  if $word$==$\{$stopword,number,punctuation$\}$ **then**
    $data_{set} = data_{corpus} - word$
  end if
end for
Token based intent extraction:
$\{unigram, , bigram, , program, tetragram\} \leftarrow n - gram(data_{set})$
$WordCount(data_{set})$
Semantic intent extraction:
$J(A, B) \leftarrow Jaccard\ Score$
$J(A, B) = \frac{\|A \cap B\|}{\|A \cup B\|} = \frac{\|A \cap B\|}{\|A\| + \|B\| - \|A \cap B\|}$
$where\ 0 \leq J \leq 1,\ '1'\ represents\ more\ similarity$
$A, B\ represents\ strings$
$b \leftarrow mean(nearest_{cluster})$
$a \leftarrow mean(intraDistance_{cluster}$
Silhouette Score$(a, b) = \frac{b-a}{max(a,b)}$
KMean$(data_{set}, Sentence_{embed})$
LDA$(data_{set}, threshold > 0.5)$
$\beta \leftarrow Extract_{Synonyms}(Intents)$
$\gamma \leftarrow Extract_{Misspellings}(Intents)$
$Training_{set} \leftarrow Max_{coverage}(\beta, \gamma)$

Fig. 2 N Gram results

```
'unigrams:'

[('phone', 13845),
 ('iPhone', 12256),
 ('iOS', 10731),
 ('update', 9713),
 ('nt', 6091),
 ('fix', 5981),
 ('Apple', 5774),
 ('battery', 5692),
 ('new', 5574),
 ('help', 4189)]

'bigrams:'

[('iPhone X', 1504),
 ('battery life', 1124),
 ('wo nt', 972),
 ('new update', 942),
 ('iPhone s', 936),
 ('Apple Music', 795),
 ('question mark', 766),
 ('iOS update', 692),
 ('new iOS', 621),
 ('new iPhone', 615)]

'trigrams:'

[('question mark box', 215),
 ('new iOS update', 159),
 ('gon na fix', 146),
 ('iPhone s Plus', 139),
 ('new iPhone X', 139),
 ('wo nt let', 138),
 ('phone keeps freezing', 126),
 ('time I type', 124),
 ('latest iOS update', 119),
 ('wo nt turn', 118)]

'tetragrams:'

[('time I type letter', 45),
 ('check overall battery health', 33),
 ('phone wo nt turn', 29),
 ('overall battery health iPhone', 25),
 ('check battery health iPhone', 23),
 ('way check overall battery', 21),
 ('la mise à jour', 19),
 ('phone wo nt update', 18),
 ('guys gon na fix', 18),
 ('way check battery health', 18)]
```

4.1 Token based intent extraction

This module focuses on extracting the keywords from the data set that occur in higher frequencies. Identifying such keywords helps to get a good coverage booster at the beginning of the pipeline itself. This is achieved through ngram analysis and word cloud analysis.

4.1.1 N-Gram analysis

A brute-force way to extract intents from the corpus is by extracting the count of uni-gram, bi-gram, tri-gram, and tetra-gram from the corpus. Stop words, numbers, and punctuations are removed from the dataset and the frequency counts of the n-gram are calculated with the help of the Spacy library [21]. The limit on the number of n-grams can be set by the analyst, although 10 seems to be a decent number. Figure 2 shows the results as a whole and we can see occurrences of some intents, for example, battery life, update issues, the phone kept freezing, etc.

4.1.2 Word cloud analysis

The word cloud diagram illustrates the most commonly occurring terms in the corpus and gives an idea of the most commonly occurring intents here. Word Cloud also helps to understand the business domain of the corpus if not understood after the ngram analysis.

Figure 3 depicts all the issues with updating, bugs, screen, etc. as they appear larger compared to other characters.

A word's size in a word cloud increases as its frequency rises. In addition, if the work focuses on a specific intent keyword that could not be detected through n-gram analysis but is present in the corpus, it should be visible in the word cloud.

The intents seen here but not shown in ngram are - users asking about bugs, itunes, time, settings, etc.

For some business domains, an intent can mean only one type of user query hence the keyword from token-based intent extraction *refund* means just *refund* intent. Hence, *refund* intent can mean just one intent in itself. But such is not always the case.

The intents like *Cancellation* are always related to some product i.e. *Cancellation of the order of iphone*,

Fig. 3 Word cloud

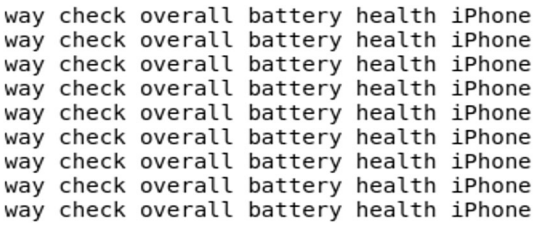

Fig. 4 Intent- check battery health

cancellation of the order of ipod etc. So, in cases where the context of the whole user query doesn't matter as in the case of *refund*, the token-based methods extract those intents with ease, and for those that need context, the semantic-based intent extraction becomes essential.

4.2 Semantic based intent extraction

Intents can be extracted from token-based analysis. For example - phone freezing, etc. But for some intent, the tokens do not give the whole context of the query asked by the user. Example - update phone. This is a frequently asked intent but the whole context of it is not known. The following sentence-based analysis helps us cluster such intents together in different ways users have asked them so that they can be successfully extracted (Fig. 4).

4.2.1 Jaccard score and Ratcliff-Obershelp algorithm

Jaccardian Score focuses on the token-based similarity between strings. Considering there are two strings A and B, the jaccardian is calculated using Eq. 1.

$$J(A,B) = \frac{\|A \cap B\|}{\|A \cup B\|} = \frac{\|A \cap B\|}{\|A\| + \|B\| - \|A \cap B\|} \quad (1)$$

The value ranges between 0 and 1 with the value towards 1 showing more similarity between the tokens of the strings.

To improve the coverage, a sequence-based string similarity approach is used - Ratcliff-Obershelp Score. It is used to find the commonality between two strings based on the common substrings in them.

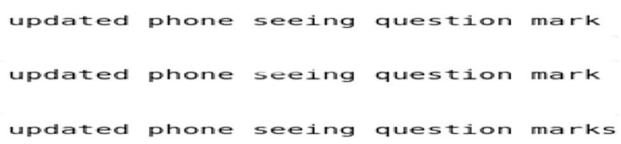

Fig. 5 Intent - question mark on updating phone

Sentences that have the same intent - can have the same tokens or differ slightly in their tokens from each other.

For example - "cannot switch on" and "not switching on" has the same intent. It has a poor Jaccard score of 0.2 but a good Ratcliff-Obershelp score of 0.81 hence both are used as a heuristic in clustering sentences together as shown in Eq. (1). To improve the accuracy of the method, a high threshold of greater than 0.8 is set for both the scores to fulfill.

From Fig. 2, *question mark* appears 766 times in the corpus but now after Fig. 5, the semantic information of it can be observed i.e. it comes after the updation of the phone. This heuristic based technique heavily depends on how sentences are formed and the position of tokens in the sentence. As a result, a paraphrased sentence meaning the same intent can be missed out, like "not switching on" and "not able to turn on". One of the reasons is due to the high threshold in the algorithm. But that helps prevent unnecessary noise from getting inside the clusters. The other paraphrased sentences meaning the same intent can be extracted from the ML algorithms as shown in the next steps of the pipeline.

4.2.2 Textacy topic modelling

This technique converts the corpus to a document term matrix with the value in the matrix demonstrating different weighted normalized values. Topic modeling techniques like Non-negative matrix factorization (NMF), Latent Dirichlet Allocation (LDA), and Latent Semantic Analysis (LSA) are applied in an attempt to get meaningful clusters [22].

The grid search method is used to tune the parameters of the function according to the highest value of the Silhouette Score [17], which can be mathematically stated as shown in Eq. 2.

$$SilhouetteScore(a,b) = \frac{b-a}{max(a,b)} \quad (2)$$

where, b = mean distance of the nearest cluster and a = mean intra distance of the cluster.

The silhouette score, Eq. 2, obtained for the corpus after the grid search is 0.46.

The grid search values in Fig. 8 that fit in the algorithm are -

1. tf type = sqrt → Type of term frequency (tf) to use for weights' local component. Weights are simply the absolute per-document tfs, i.e. value (i, j) in an output doc-

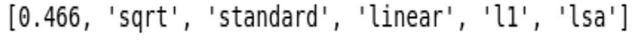

Fig. 6 Grid search results for textacy topic modelling

Fig. 7 Clusters formed by topic modelling

```
topic 0 : update   fix   phone   ios   iPhone   battery   new   issue   work   iOS
topic 1 : fix   glitch   y   issue   letter   shit   go   problem   need   bug
topic 2 : iPhone   ios   battery   plus   drain   s   fix   X   issue   life
topic 3 : type   help   iPhone   letter   happen   try   question   work   time   plus
topic 4 : type   letter   update   question   mark   ios   box   iOS   happen   new
topic 5 : ios   battery   help   drain   phone   bug   iphone   type   life   fast
topic 6 : work   ios   happen   app   yes   type   bug   stop   try   issue
topic 7 : help   update   issue   need   app   problem   ios   happen   have   late
topic 8 : ios   question   mark   phone   iPhone   bug   box   plus   y   shit
topic 9 : happen   app   phone   type   freeze   bug   fix   time   go   keep
```

term matrix corresponds to the number of occurrences of term j in doc i.

2. idf type = standard → Type of inverse document frequency (idf) to use for weights' global component. Terms appearing in many docs have higher document frequencies (dfs), correspondingly smaller idfs, and in turn, lower weights.
3. dl type = linear → Type of document-length scaling to use for weights' normalization component
4. norm = l1 → Normalize weights by the L1 norm of row-wise vectors
5. model = lsa → Topic modelling model chosen to give highest silhouette score

The score varies between -1 and 1 with near 0 values denoting overlapping clusters and near 1 denoting perfectly separated clusters. From Figs. 6 and 7, the following intents of drainage in battery, a question mark appearing strangely in the phone which may be a bug, freezing of the phone, update issues happening in iOS lately, etc. can be observed. This helps to build contexts from the intent keyword. For example - where and why is 'update issue' being used in user queries etc. To cover a larger corpus, the next steps in the pipeline are considered as they involve new sentences in the clusters giving a better and more idea of the context in which the intents are framed. They boost the use cases in which the intents are used through the use of sentence embeddings.

4.2.3 KMeans algorithm using sentence embeddings

Embeddings convert the word sentences into vector representation over the vector of space of default dimension. Usually, 300 dimensions are considered to be used by researchers as they are capable of getting around enough semantic information from the data [23, 24].

The vectorial representation i.e. embeddings includes the semantic information of the sentences and the assumption here is that the average of embeddings of words in a sentence of the same intents should almost be equal as their semantic information is the same. The word embeddings of the sentences with the same intent would lie close to each other on the vector space. Therefore, the common intents should be

```
model_z.similar_by_word("battery")
```

```
[('hours', 0.9999765157699585),
 ('updating', 0.999974250793457),
 ('slow', 0.9999676942825317),
 ('charge', 0.9999645948410034),
 ('ve', 0.999963104724884),
 ('music', 0.9999586343765259),
 ('apps', 0.999958336353302),
 ('stuck', 0.9999582767486572),
 ('restart', 0.9999572038650513),
 ('s', 0.9999572038650513)]
```

Fig. 8 Word embedding example - battery

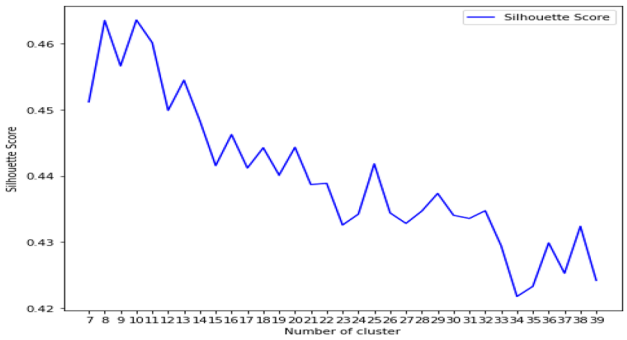

Fig. 9 Silhouette score - finding optimal clusters

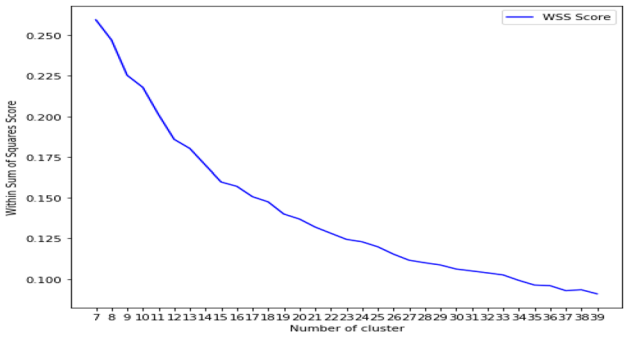

Fig. 10 Elbow curve - finding optimal clusters

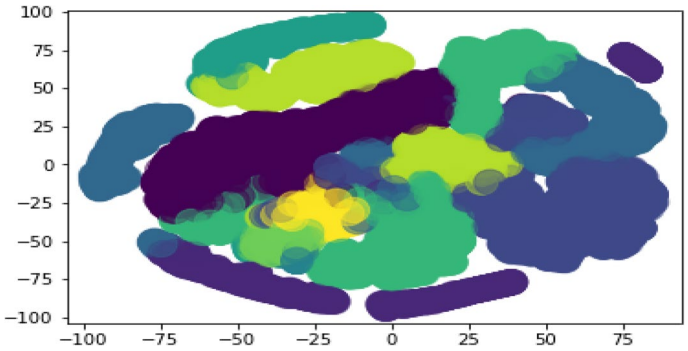

Fig. 11 T-score of sentence embeddings

```
'updated phone freezes lot tap app open properly freezes crashes',
'Dowloaded iOS iphone  Phone freezes  hours force restart   OK',
'phone freezes regular basis regret iOS  billion phones work',
'random phone freezes  phone related issues rest time  access apps glass screen respond touch',
'continually freezes running multiple apps single apps half time wo switch speaker phone selected',
'ios iPhone  freezes glitches battery life  Siri listen Help',
'phone like  second delay input response input Idk happening type phone freezes unfreezes typing happens super f
ıst Help aaaaaa',
'downloading IOS  iPhone S slower slower freezes time know',
'new OS TERRIBLE freezes locks phone constantly issues shocking released',
'iPhone s runs ios freeze apple logo need restore reinstalling  GB operating system',
'iPhone X screen regularly freezes cold sure specific iPhone X iOS  general nt notice iPhone  iOS',
```

Fig. 12 Scoping down on a cluster formed by KMeans - sentence embedding

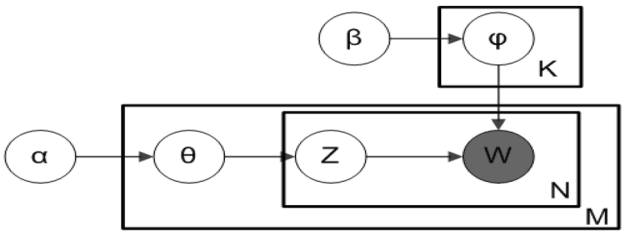

Fig. 13 LDA topic modelling

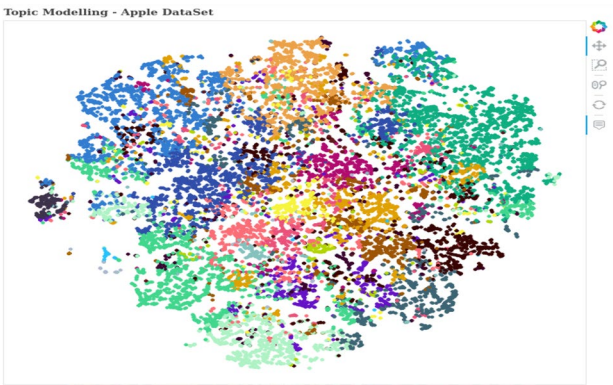

Fig. 14 LDA - without threshold

clustered through this procedure. In Fig. 8, it is seen that the words lie in the semantic neighborhood of the word battery.

To get the optimum number of clusters [25] two plots have been plotted indicating the maximum silhouette score that can be achieved and the corresponding elbow curve shown in Figs. 9 and 10 respectively. In the silhouette curve Fig. 9, the number of clusters where the score is highest to 1 is looked at. It can be observed that the graph peaks at 10 clusters (Fig. 11).

From the elbow curve Fig. 10, an elbow occurs between 9 and 10. This confirms that optimum results can be achieved by setting the number of clusters as 10. Hence, we should achieve a minimum of 10 distinct intents from the dataset. The clusters can then be scoped down to get further results as shown in work 12. The focus in Fig. 12 is on the intent "phone freezing" and different queries related to it can be observed through the cluster that we achieved through sentence embeddings.

The clusters for the 300 dimension vectors in KMeans Sentence Embeddings can be illustrated in 2 dimensions using the t-score method. A clear cluster separation is observed in Fig. 13 based on 10 intents showing that the corpus will give us a minimum of 10 meaningful separate clusters from it. LDA topic modeling is used for better clustering and a richer experience. For clustering queries, a threshold of 0.5 is used for similarity scores.

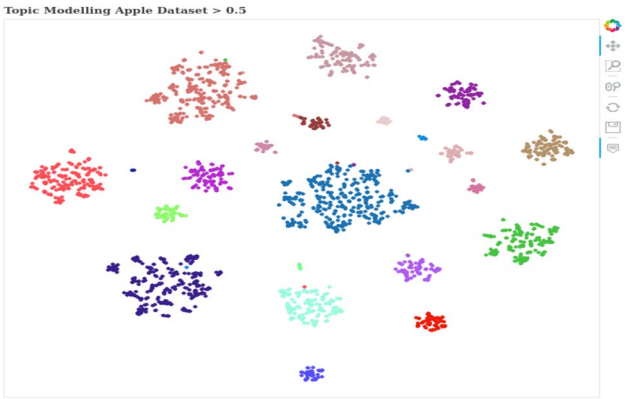

Fig. 15 LDA - with threshold

4.2.4 LDA topic modelling

LDA topic modeling is illustrated algorithmically in Fig. 13.

Here, M = number of documents i.e. queries in the dataset
N = number of words per topic/cluster
K = number of topics/cluster

Dir(α) is used to denote the topic distribution of a certain document given by θ. From θ, a specific topic is picked up. Dir(β) gives us the word distribution called ϕ for a certain topic. From this distribution, a word is selected. Hence, then the word is assigned to a topic. This method is extensively used in the clustering of sentences with the same semantic information. In Fig. 14, bokeh plots help in obtaining well-defined interactive clusters though there is an overlap between the plots. The plot consists of user queries in different colors depicting different intents. There is a huge overlap in the graph and needs to be separated to understand the queries better. Hence, to remove the overlap in the center a probability threshold of greater than 0.5 is introduced. This gives separated and well-defined clusters. Visualization of the separated clusters through the plot can be seen in Fig. 15.

The probability threshold of greater than 0.5, helps to get well-defined clusters as only those sentences in the topics that have a probability of greater than 0.5 get clustered in the topic. Hence, this reduces the noise in the clusters, and the sentences in the clusters can be extracted by hovering the mouse over the graph on the bokeh plot. After this, the next step is to find what keywords of intent are used commonly together in queries. For this work, the pyLDAvis representation is shown in Sect. 4.2.5, [26].

4.2.5 pyLDAvis representation

This library is used for breaking down information present inside a cluster and extracting the top words present in the clusters. This helps to find the relevance of specific terms and the tf inside the cluster. It can be stated that overlapping between clusters can be depicted and understood through this method. The method is used to understand the trend of the corpus. The circles represent the size of the clusters with the cluster number mentioned on the circles. Selecting each circle gives us the top - terms present in descending order as well as their percentage composition in the cluster.

In Fig. 16, a random cluster number 4, has been taken as an example to demonstrate the library. The cluster illustrates the problems with the phone and recognizes the top keywords from that cluster. A few keywords like - update, freeze, lagging, dead, etc can be obtained in the same cluster meaning that they have been used together in the queries asked by the user. Hence, before training, it is necessary to

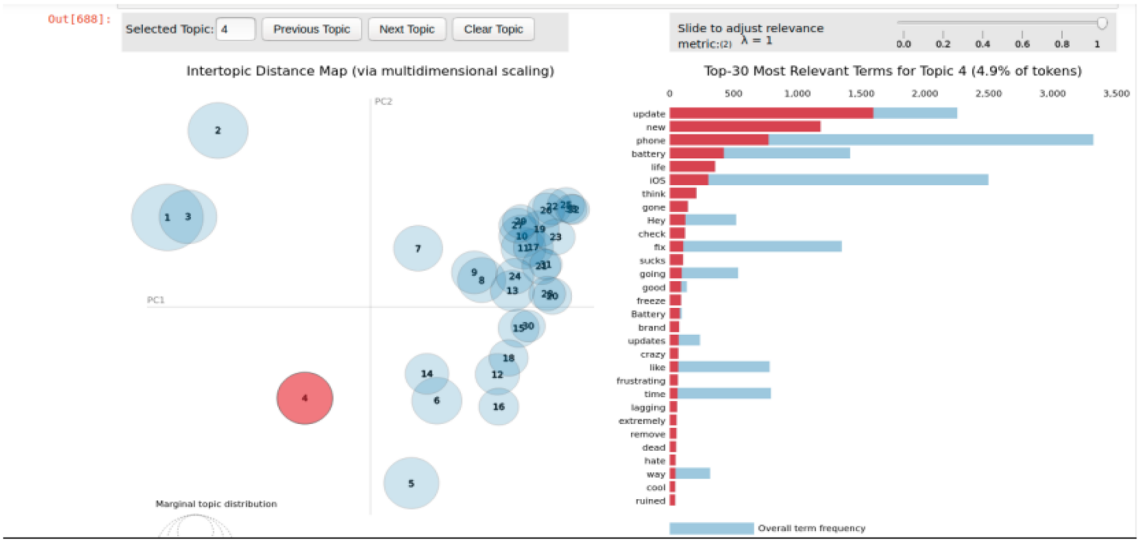

Fig. 16 pyLDAvis representation of the cluster

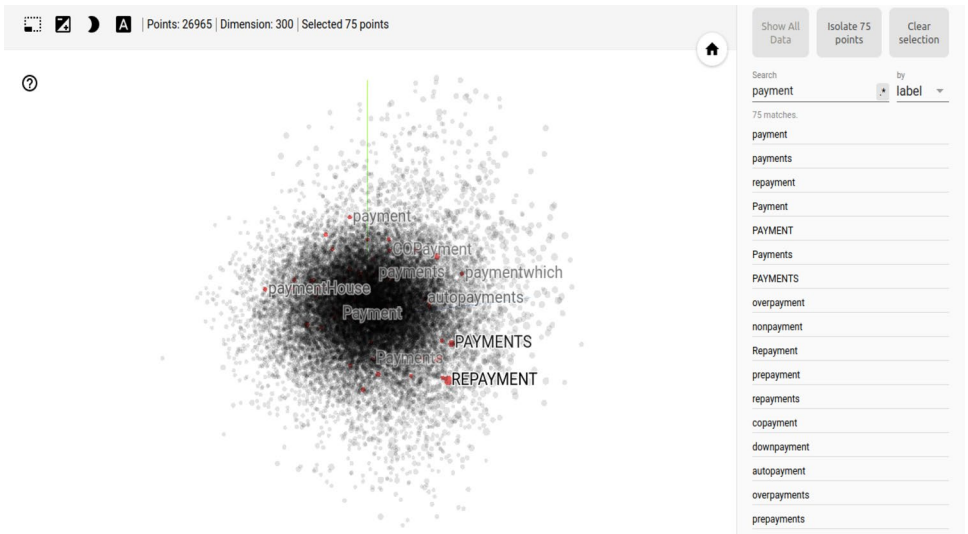

Fig. 17 Extracting synonyms - 'payment'

introduce or cross verify with these keywords occurrences whether some keywords are missed that can be clustered together with intent, for example, queries with lagging can be clubbed together in freezing of phone, etc.

4.3 Maximising coverage

After obtaining the intents from the above token and sentence extraction techniques, it is important to focus on improving the coverage i.e. to cover maximum queries in the dataset before feeding to a classifier ML model. Intents can be paraphrased in different ways and need to identify those paraphrases. One of the paraphrases is to use the word with a similar meaning to that of the intent in the customer query i.e. synonyms of intents. According to Fig. 17 it can be realized that payment can be spelled as repayment, overpayment, etc. based on the context of the query, and it's essential to pick these up under the intent payment. Hence, such paraphrases need to be identified and categorized under one intent. To get the final coverage boost, the synonyms, and misspellings of the intent are covered considering the dataset. In a corpus, for example, the intent updation can be spelled as updat, updet, etc. Both of these words have to be picked up for coverage boost.

4.3.1 Extracting synonyms of intents

The study [27], learns vectors only for complete words found in the training corpus. Word2vec text vector [27] takes a word as a minimum input [28]. So, synonyms can easily be extracted from the main intent words by the representation of the word vector on the Tensorboard using Tensorflow [29, 30]. The closest cosine value vectors would be the synonyms of the target intent. One important aspect to be ensured is

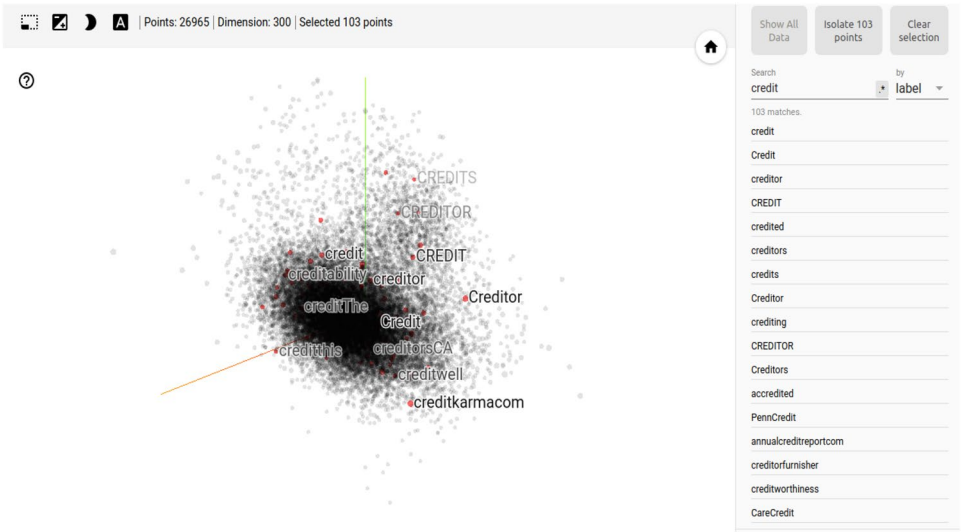

Fig. 18 Extracting misspellings - 'credit'

that there should be no null values before training the model for better results. In Fig. 17, we have found the synonyms used in the corpus for the intent - 'payment'.

4.3.2 Extracting misspellings of intents

FastText [31] forms embeddings based on the subword approach. Hence, different subwords yield different misspellings of the intent that can be visualized on the Tensorboard through Tensorflow [29, 30]. The closest cosine value vectors would be the misspellings of the target intent. The benefit for programmers here is to not check for null values as it takes such into consideration as well. In Fig. 18, the work has found the misspellings used in the corpus for the intent - 'credit'.

4.4 Prioritising intent pickup

After passing through all the above methods, the intents that have the highest coverage with minimum overlap with other intents should be picked up first to give the coverage boosts at the earliest. This is achieved through a library called Snorkel, which helps us prioritize the intent pickup order, thorough coverage, and overlap columns as shown in Table 1.

Snorkel is the library used to understand the overlapping present between the intents through its labeling functions and the coverage of each intent in the entire dataset. The intent with the maximum coverage can be taken as a priority for the classifier to be trained on it so that the highest coverage per intent boost is received early. If there is not much difference in overlapping, intent with maximum coverage is considered. For this proposed work three intents of the dataset have been chosen - freezing of the phone, battery-related issues, and button-related problems as an example in Table 1.

It can be observed that freezing of the phone is the intent with the maximum coverage followed by battery problems and button-related problems. As there is minimal overlap between them, this order would be picked up to get the maximum coverage boost at the earliest (Fig. 19).

Table 1 Snorkel coverage and overlap of each intent

Intents	j	Polarity	Coverage	Overlaps	Conflicts
Regex phone freeze related	0	1	1.145643	0.012188	0.012188
Regex battery problem related	1	2	0.877514	0.018282	0.018282
Regex button related keywords	2	3	0.079220	0.012188	0.012188

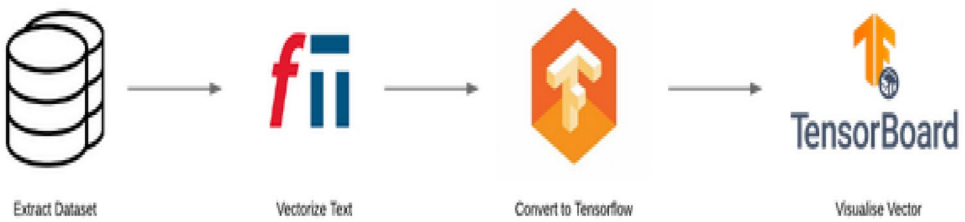

Fig. 19 Extraction synonyms and misspellings from the corpus

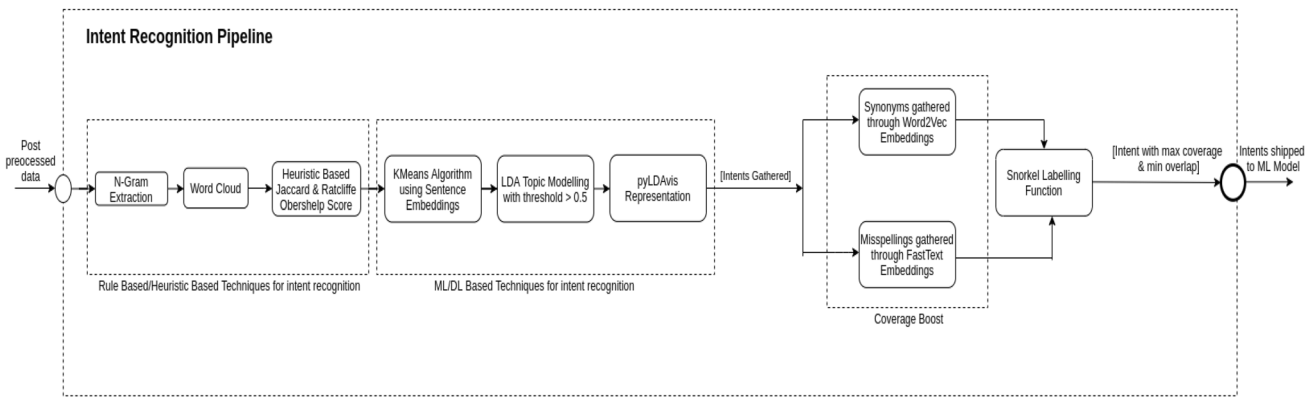

Fig. 20 Complete intent recognition pipeline

5 Built intent recognition pipeline

The complete exploration pipeline built on the above methods is illustrated in Fig. 20. It uses heuristic as well as machine learning approaches mentioned above to give a good coverage boost as well as a proper understanding of the dataset. It is the combination of token-based intent extraction, sentence-based intent extraction, maximizing coverage, and prioritizing intent order one after the other as explained in section IV.

6 Evaluation

The pipeline streamlined intent discovery and it has been observed that even if one of the components fails to deliver a result, its outcome is covered either in the previous or the next component, leading to minimal result variations or dependence on one of the components. Therefore, there is a fallback mechanism in place as a result of overlapping/non-overlapping results between different levels of the pipeline which ensures that none of the intents are missed.

It was observed that the extraction of synonyms and misspellings for different intents helped us see a coverage jump. The machine learning model wasn't able to cover after its first run of prediction as extracting misspellings and paraphrased sentences of the intents was not done before in the training dataset.

The synonyms and misspelling pipeline can be separated from the above exploration pipeline as shown in Fig. 19. FastText vectors capture hidden information about a language, like word analogies or semantic information. The word analogies and semantic information help in:

1. Finding and extracting misspellings of a target word in the entire dataset.
2. Finding words that occur commonly in context (in neighbor) to the target word.
3. Finding similarity between different words given as a cosine of the angle between the word vectors.

These techniques contributed to preparing a training set worthy of maximizing coverage and hence we received a coverage boost of 10–11% is observed as the ML model predicted queries that it hadn't covered before.

7 Conclusion

A quick and efficient intent recognition pipeline has been built to recognize intents based on a dataset without filtering. It is built on three steps - intent extraction, maximizing coverage by identifying synonyms and misspellings of the intents, and deciding the order in which intents are to be trained by recognizing their overlap and coverage in the dataset. The first step involves extracting intents from the dataset using heuristics and machine learning. The techniques are independent of the business domain of the corpus. These steps include token based intent clustering and sentence based intent clustering. The second step deals with synonyms and misspellings of the intents picked up from the first approach in a bid to increase the training dataset and make a conversational AI respond to diverse queries related to the intents it is trained on. In the third step, the pipeline concludes with the use of Snorkel to include intents in the classification model in the decreasing order of their coverage on the dataset. The pipeline does not look at data before picking up intents, so there is no bias. This is the key factor defining the success of the pipeline. In addition to generalizing over multiple business domains, the system can be used effectively to increase coverage or improve understanding of the corpus.

Future improvements can be made to the pipeline by training it on more diverse business data or data from other industries like travel, finance, healthcare, etc. The future scope of the paper would involve understanding the performance of each component in the pipeline. The pipeline can then be tweaked by adding or removing steps to identify intents at an even quicker pace. We can thereby try to figure out the overlap of intents that can be detected from each step in the pipeline and hence, remove redundant steps from the pipeline. Similar intent detection can also be used to identify critical queries that require immediate attention, especially in healthcare or finance industries. For example, if we have a healthcare query - "There is an emergency with patient X" or a finance query - "I mistakenly shared my ATM PIN with a hacker" we need to quickly identify the query even if there is a misspelling or synonym in the query. Use cases like these can be covered in the future based on the pipeline we have and can be built on top of it .

Funding Open access funding provided by Manipal Academy of Higher Education, Manipal.

Data availability Data is obtained from Kaggle dataset of Customer Support on Twitter at https://www.kaggle.com/datasets/thoughtvector/customer-support-on-twitter

Declarations

Conflict of interest The authors declare no conflict of interest

References

1. Mohit M (2020) String similarity-the basic know your algorithms guide! https://itnext.io/string-similarity-the-basic-know-your-algorithms-guide-3de3d7346227, Medium, ITNEXT. Accessed 8 Jan 2022
2. Fernandes A (2020) 7 definitive AI chatbot trends for 2019. https://blog.verloop.io/chatbot-applications-top-10-industries-that-use-chatbots/. Accessed 5 July 2022
3. Blei DM, Ng AY, Jordan MI (2023) Latent dirichlet allocation. J Mach Learn Res 3:993–1022
4. Guo S, Yao N (2021) Document vector extension for documents classification. IEEE Trans Knowl Data Eng 33(8):3062–3074. https://doi.org/10.1109/TKDE.2019.2961343
5. Rohit B, Exploration and visualisation of word vectors in chat, text vector visualisation. https://rohetoric.github.io/text-vector-visualisation/. Accessed 10 Oct 2021
6. Oanh Thi T, Tho Chi L (2020) Understanding what the users say in chatbots: a case study for the Vietnamese language. Eng Appl Artifiv Intell 87:103322. https://doi.org/10.1016/j.engappai.2019.103322
7. Liu Jiao, Li Yanling, Lin Min (2019) Review of intent detection methods in the human-machine dialogue system. J Phys: Conf Ser 1267:012059. https://doi.org/10.1088/1742-6596/1267/1/012059
8. Kapočiūtė-Dzikienė J, Balodis K, Skadiņš R (2020) Intent detection problem solving via automatic DNN hyperparameter optimization. Appl Sci 10:7426. https://doi.org/10.3390/app10217426
9. Kathuria Ashish, Jansen Jim, Hafernik Carolyn, Spink Amanda (2010) Classifying the user intent of web queries using k -means clustering. Internet Res 20:563–581. https://doi.org/10.1108/10662241011084112
10. Jansen Bernard J (2006) Spink Amanda how are we searching the world wide web? A comparison of nine search engine transaction logs. Inf Process Manag 42(1):248–263. https://doi.org/10.1016/j.ipm.2004.10.007
11. Ratner A, Bach SH, Ehrenberg HR, Fries JA, Wu S, Ré C (2017) Snorkel: rapid training data creation with weak supervision. Proc VLDB Endow 11(3):269–282. http://arxiv.org/abs/1711.10160. Accessed 8 Jan 2022
12. Snorkel Intro Tutorial: Data Labeling. https://www.snorkel.org/use-cases/01-spam-tutorial. Accessed 2 Apr 2022
13. Visvam Devadoss AK, Thirulokachander VR, Visvam Devadoss AK (2019) Efficient daily news platform generation using natural language processing. Int J Inf Technol 11:291–311. https://doi.org/10.1007/s41870-018-0239-4
14. Myint STY, Sinha GR (2019) Disambiguation using joint entropy in part of speech of written Myanmar text. Int J Inf Technol 11:667–675. https://doi.org/10.1007/s41870-019-00336-4
15. Gopi AP, Jyothi RNS, Narayana VL, Sandeep KS (2023) Classification of tweets data based on polarity using improved RBF kernel of SVM. Int J Inf Technol 15:965–980. https://doi.org/10.1007/s41870-019-00409-4
16. Sintayehu H, Lehal GS (2021) Named entity recognition: a semi-supervised learning approach. Int J Inf Technol 13:1659–1665. https://doi.org/10.1007/s41870-020-00470-4
17. Thukral A, Dhiman S, Meher R, Bedi P (2023) Knowledge graph enrichment from clinical narratives using NLP. NER, and biomedical ontologies for healthcare applications. Int J Inf Technol 13:53–65. https://doi.org/10.1007/s41870-022-01145-y
18. Alejandro F, John A (2016) Ensembling classifiers for detecting user intentions behind web queries. IEEE Internet Comput 20(2):8–16. https://doi.org/10.1109/MIC.2015.22
19. Joyce X (2018) Topic modeling with LSA, PSLA, LDA & lda2Vec, medium, NanoNets. https://medium.com/nanonets/37topic-modeling-with-lsa-psla-lda-and-lda2vec-555ff65b0b05. Accessed 20 Dec 2022
20. Customer Support on Twitter. https://www.kaggle.com/thoughtvector/customer-support-on-twitter. Accessed 2 Dec 2022
21. Honnibal M, Montani (2017) spaCy 2: natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing
22. Burton DeWilde (2023) textacy: Nlp, before and after spacy, GitHub. https://chartbeat-labs.github.io/textacy/build/html/index.html. Accessed 5 Jan 2020
23. Ding W, Zhang Y, Sun Y, Qin T (2021) An Improved SFLA-Kmeans algorithm based on approximate backbone and its application in retinal fundus image. IEEE Access 9:72259–72268. https://doi.org/10.1109/ACCESS.2021.3079119
24. Abusubaih MA, Khamayseh S (2022) Performance of machine learning-based techniques for spectrum sensing in mobile cognitive radio networks. IEEE Access 10:1410–1418. https://doi.org/10.1109/ACCESS.2021.3138888
25. Rousseeuw PJ (1987) Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. J Comput Appl Math 20:53–65
26. Sievert C, Shirley K (2014) LDAvis: a method for visualizing and interpreting topics. In: Proceedings of the workshop on interactive language learning, visualization, and interfaces. Association for Computational Linguistics, Baltimore, MD, USA, pp 63–70
27. Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. http://arxiv.org/abs/1301.3781
28. K means clustering example with word2vec in data mining or machine learning. https://ai.intelligentonlinetools.com/ml/k-means-clustering-example-word2vec/. Accessed 5 Feb 2020
29. Goldsborough P (2016) A tour of TensorFlow [Online]. https://arxiv.org/pdf/1610.01178.pdf. Accessed 3 Feb 2022
30. Abadi M, Barham P, Chen J, Chen Z, Davis A, Dean J, Devin M, Ghemawat S, Irving G, Isard M, Kudlur M, Levenberg J, Monga R, Moore S, Murray DG, Steiner B, Tucker P, Vasudevan V, Warden P, Wicke M, Yu Y, Zheng X (2016) Tensorflow: a system for large-scale machine learning. In: 12th USENIX symposium on operating systems design and implementation (OSDI 16), pp 265–283. https://www.usenix.org/system/files/conference/osdi16/osdi16-abadi.pdf. Accessed 4 Jan 2022
31. Joulin A, Grave E, Bojanowski P, Mikolov T (2016) Bag of tricks for efficient text classification. http://arxiv.org/abs/1607.01759. Accessed 23 Jan 2022